Exact Stanley-Depth Certificates for Ten Squarefree Veronese Cases

KEIO

26 October 2026

Abstract

We give explicit finite interval partitions proving $\operatorname{sdepth}(I_{14,2}) = 6$, $\operatorname{sdepth}(I_{17,2}) = 7$, and $\operatorname{sdepth}(I_{19,3}) = 7$ over every field. The established variable-splitting recurrence then determines $\operatorname{sdepth}(I_{n,2})$ for $14 \leq n \leq 19$ and $\operatorname{sdepth}(I_{n,3})$ for $19 \leq n \leq 22$. These are ten parameter cases of one general Stanley-depth conjecture, not ten separately resolved conjectures. All three certificates, encoded by cyclic interval representatives, and a complete standard-library verifier are embedded in this single source. Independent mask-by-mask checks validate the expanded partitions. The cases lie outside the explicit ranges of the primary papers examined; historical priority is not established. The general conjecture remains unresolved by this work. The proof search adapts a strategy of handling forced low-rank objects before completing a disjoint cover; its algebraic correspondence, counting bound, and recurrence are established tools rather than new principles.

1 The precise external problem and the result

Let $S = \mathbb{K}[x_1, \ldots, x_n]$, where $\mathbb{K}$ is any field, and let

$$I_{n,d} = (x_{i_1} \cdots x_{i_d} : 1 \leq i_1 < \cdots < i_d \leq n).$$

A Stanley decomposition is a direct sum of multigraded $\mathbb{K}$-vector spaces $I = \bigoplus_j u_j \mathbb{K}[Z_j]$, with each summand free over the polynomial ring on the variable set Z_j. Its depth is $\min_j |Z_j|$, and $\operatorname{sdepth}(I)$ is the maximum of these depths. The squarefree Veronese formula conjecture predicts

$$\operatorname{sdepth}(I_{n,d}) = d + \left\lfloor \frac{n-d}{d+1} \right\rfloor. \tag{1}$$

This is the conjecture discussed by Cimpoeaş and Keller–Shen–Streib–Young [1], and subsequently by Ge–Lin–Shen [2]. It concerns Stanley depth. The known formula for Hilbert depth does not establish it [5, Remark 3.11].

Theorem 1.1. *Over every field,*

d	n	$\operatorname{sdepth}(I_{n,d})$
2	$14, 15, 16$	6
2	$17, 18, 19$	7
3	$19, 20, 21, 22$	7

The result is a finite-case confirmation of (1). It is not a resolution for every n, d, nor a resolution of the separate general inequality $\operatorname{sdepth}(M) \geq \operatorname{depth}(M)$, which is false for general multigraded modules. No WOWII numbered entry is added by this result.

2 From interval partitions to a complete proof

Write $P_{n,d} = \{A \subseteq [n] : |A| \geq d\}$ and $[A, B] = \{U : A \subseteq U \subseteq B\}$.

Lemma 2.1 (Classical interval correspondence)**.** *If $P_{n,d}$ is partitioned into intervals $[A, B]$ with $|B| \geq s$, then* $\operatorname{sdepth}(I_{n,d}) \geq s$.

Proof. For each interval take the Stanley space $x_A\mathbb{K}[x_i : i \in B]$, where $x_A = \prod_{i\in A} x_i$. Its monomials have precisely the supports in $[A, B]$. Every monomial of $I_{n,d}$ has a support in exactly one interval and lies in the corresponding space. Thus these spaces form a direct sum equal to the entire ideal, including all nonsquarefree monomials. Every space has at least s free variables. □

This direction of the Herzog–Vlădoiu–Zheng correspondence is recalled explicitly in [1, Section 2.1]; see also [3].

Lemma 2.2 (Counting upper bound)**.** *For $1 \leq d \leq n$,*

$$\operatorname{sdepth}(I_{n,d}) \leq d + \left\lfloor \frac{n-d}{d+1} \right\rfloor.$$

Proof. In any Stanley decomposition of depth s, each squarefree degree-d monomial belongs to a summand whose generating monomial is exactly that monomial: there are no smaller-degree monomials in the ideal. At least $s - d$ of its free variables are outside its support. Multiplying by these variables supplies at least $s - d$ distinct squarefree degree-$(d + 1)$ monomials. Directness makes all the resulting monomials distinct between summands. Hence

$$\binom{n}{d}(s-d) \leq \binom{n}{d+1}.$$

Taking an integer upper bound proves the claim. If $s \leq d$, it is immediate. □

This is the established bound of [1, Lemma 2.2].

2.1 The three explicit certificates

Identify $[n]$ with $\mathbb{Z}/n\mathbb{Z}$ by shifting the labels by one. An integer mask encodes a subset: bit i denotes x_{i+1}. For each seed (A, B) in Appendix A, rotate both masks through all n cyclic shifts and keep the distinct resulting intervals. The data have the following exact sizes.

(n, d)	Top rank	Seeds	Nontrivial intervals	All intervals	$\lvert P_{n,d}\rvert$
$(14, 2)$	6	92	1274	12911	16369
$(17, 2)$	7	472	8024	109294	131054
$(19, 3)$	7	969	18411	480492	524097

For the first case the lower-rank counts are $91, 455, 728$ at ranks $2, 4, 5$. For the second they are $136, 1020, 1768, 5100$ at ranks $2, 4, 5, 6$. For the third they are $969, 5814, 11628$ at ranks $3, 5, 6$. All nontrivial upper endpoints have the displayed target rank.

The verifier checks endpoint validity, distinctness of seed orbits, pairwise disjointness of expanded intervals, and coverage of every subset with rank between d and the target minus one. Every still uncovered higher-rank subset is assigned its singleton interval. These checks therefore establish a partition of all of $P_{n,d}$. Lemma 2.1 proves the lower bounds $6, 7, 7$, and Lemma 2.2 makes them equalities.

This is an exact finite proof certificate. It is not a random test of the conjecture, and it does not require trusting an optimization solver's success report. The full data and a checker are in the paper; the same program is provided separately as `certificates.py`. The certificate establishes the assertion over every field because the support partition is independent of its coefficients.

2.2 Propagation to ten parameter cases

Lemma 2.3 (Established variable splitting)**.** *For $n > d \geq 2$,*

$$\operatorname{sdepth}(I_{n,d}) \geq \min\{\operatorname{sdepth}(I_{n-1,d}), 1 + \operatorname{sdepth}(I_{n-1,d-1})\}.$$

Proof. By the full Herzog–Vlădoiu–Zheng correspondence [3], choose interval partitions attaining the two Stanley depths on the right. Partition supports not containing n using that partition of $P_{n-1,d}$. Partition supports containing n by adjoining n to both endpoints of each interval in a partition of $P_{n-1,d-1}$. These two families are disjoint and cover the required supports. Their minimum top ranks give the bound. □

This is the standard recurrence from [1, Lemma 2.5]. We also use the established maximal-ideal formula $\operatorname{sdepth}(I_{m,1}) = \lceil m/2 \rceil$ [4].

Proof of Theorem 1.1. The three base values follow from the verified certificates above. Starting with $\operatorname{sdepth}(I_{14,2}) = 6$, the recurrence and maximal-ideal formula imply $\operatorname{sdepth}(I_{m,2}) \geq 6$ for every $m \geq 14$. Starting with $\operatorname{sdepth}(I_{17,2}) = 7$ gives $\operatorname{sdepth}(I_{m,2}) \geq 7$ for every $m \geq 17$. The upper bound equals six at $14 \leq n \leq 16$ and seven at $17 \leq n \leq 19$, proving the first two rows. Finally start with $\operatorname{sdepth}(I_{19,3}) = 7$. The $d = 3$ recurrence needs only $\operatorname{sdepth}(I_{m,2}) \geq 6$, already established for all $m \geq 14$. It therefore gives $\operatorname{sdepth}(I_{n,3}) \geq 7$ for every $n \geq 19$. The upper bound is seven at $19 \leq n \leq 22$, proving the last row. □

3 What the transfer contributes

The earlier WOWII work used forced extremal cases, selected low-degree configurations, and disjoint certificates to turn local constraints into global bounds. Here the analogous objects are the bottom ranks of a Boolean lattice. At the boundary cases $(14, 2), (17, 2), (19, 3)$, attaining the counting upper bound exhausts the next rank exactly. We first enforce this saturation, then cover the remaining lower ranks by disjoint intervals. Cyclic symmetry reduces the search to interval orbits. An exact-cover solver found the witnesses; the delivered checker has no solver dependency.

This is an adaptation of a proof-search strategy to commutative algebra. The graph domination inequalities do not themselves imply the Stanley-depth result. The algebraic bridge, interval-partition method, symmetry reduction, counting bound and recurrence have antecedents; no claim of a wholly new method is made. The concrete output is the three explicit partitions and their ten-case consequence.

3.1 Verification and precise priority scope

The embedded checker exhaustively validates all 2^{14}, 2^{17} and 2^{19} masks. Independent code, without cyclic-orbit logic, checks each Boolean mask against the full expanded intervals and its singleton entry. The two smaller cases were independently checked in Python; the largest was checked in C++20 with integer bit operations. All three checks passed. The research archive includes these programs, expanded and compressed certificates, and their outputs. No formal proof-assistant verification or external refereeing is claimed.

Keller et al. prove (1) for $n < 5d + 4$ and report computational confirmation through $n = 13$ [1]. The exact range in Ge–Lin–Shen [2] ends at $n = 10$ for $d = 2$ and $n = 14$ for $d = 3$, and their further lower bounds do not give the new displayed equalities. The September 2025 version of Bălănescu–Cimpoeaş–Krattenthaler [5, Remark 3.11] retains the general Stanley-depth formula as a conjecture and explicitly distinguishes it from the proved Hilbert-depth formula. All ten cases are outside those explicit exact ranges. Targeted searches located no earlier exact proof of the three base cases, but cannot rule out every published or unpublished computation. Consequently historical priority is left open.

4 Other attempted transfers and what remains

Parallel attempts considered the union-closed sets conjecture, the 1/3–2/3 order conjecture, geometric Steiner-ratio problems, and fair-allocation and discrepancy problems. None is claimed solved here. The accompanying research notes give exact obstructions to some natural transfers: inclusion-preserving matchings need not exist even when every element is abundant in a union-closed family; deleting a simplicial incomparable element need not preserve a chosen balanced pair; and an incidence-graph maximum cut can have arbitrarily bad discrepancy. These are limitations of particular arguments, not counterexamples to the named conjectures. A sharp continuous metric-tree inequality and a shape-stability proof are also retained in the notes, with no priority claim.

The positive conclusion of this paper is precisely Theorem 1.1. The general squarefree Veronese conjecture, including all larger parameter values not covered by the cited theorems or the stated recurrence bounds, is not settled.

References

[1] M. T. Keller, Y.-H. Shen, N. Streib and S. J. Young, On the Stanley depth of squarefree Veronese ideals, *Journal of Algebraic Combinatorics* 33 (2011), 313–324. https://arxiv.org/abs/0910.4645.

[2] M. Ge, J. Lin and Y.-H. Shen, On a conjecture of Stanley depth of squarefree Veronese ideals, https://arxiv.org/abs/0911.5458, version 3 (2010).

[3] J. Herzog, M. Vlădoiu and X. Zheng, How to compute the Stanley depth of a monomial ideal, *Journal of Algebra* 322 (2009), 3151–3169. https://doi.org/10.1016/j.jalgebra.2008.01.006.

[4] C. Biró, D. M. Howard, M. T. Keller, W. T. Trotter and S. J. Young, Interval partitions and Stanley depth, *Journal of Combinatorial Theory, Series A* 117 (2010), 475–482. https://doi.org/10.1016/j.jcta.2009.07.008.

[5] S. Bălănescu, M. Cimpoeaş and C. Krattenthaler, On the Hilbert depth of monomial ideals, https://arxiv.org/abs/2306.09450, version 5, 11 September 2025.

A Complete certificate data and verifier

Copy the following listing verbatim into `certificates.py` and run it with ordinary Python 3.10 or later. It reads no files and needs only the standard library. Each case ends in `PASS` after every subset has been checked. The hexadecimal integers are the actual proof data, not a pseudorandom seed or an instruction to rerun the search.

```
"""Exact finite proof certificates; Python 3.10+, standard library only.

Run: python certificates.py
No optimization solver, external files or numerical tolerances are used.
Bit i denotes variable x_(i+1). Every interval seed is rotated cyclically.
"""
from collections import Counter
from math import comb
import json

CASES = [
    (14, 2, 6, [
        (0x3, 0x327), (0x5, 0x2345), (0x9, 0x242d), (0xf, 0x220f),
        (0x11, 0x7d), (0x17, 0x237), (0x21, 0xb25), (0x2b, 0x83b),
        (0x2f, 0x6f), (0x33, 0x20b3), (0x41, 0x244b), (0x47, 0xd7),
        (0x4d, 0x46d), (0x4f, 0x24f), (0x63, 0xc63), (0x65, 0x2165),
        (0x69, 0x20e9), (0x6b, 0xeb), (0x73, 0x7b), (0x81, 0x68d),
        (0x8f, 0x28f), (0x95, 0x8b5), (0x9b, 0xbb), (0x9d, 0xbd),
        (0xa3, 0x22a3), (0xa5, 0x20a7), (0xa9, 0x30a9), (0xab, 0x4ab),
        (0xad, 0x2ad), (0xb9, 0xf9), (0xc5, 0x1e5), (0xc9, 0x1cd),
        (0xd5, 0xf5), (0xe3, 0x1e3), (0xf1, 0x10f1), (0x10b, 0x32b),
        (0x10d, 0x211d), (0x10f, 0x18f), (0x113, 0x13b), (0x115, 0x317),
        (0x119, 0x1139), (0x12d, 0x16d), (0x131, 0x1331), (0x135, 0x335),
        (0x149, 0x34d), (0x151, 0x1351), (0x155, 0x157), (0x169, 0x369),
        (0x171, 0x173), (0x183, 0x993), (0x185, 0xb85), (0x187, 0x1187),
        (0x189, 0x19b), (0x18d, 0x98d), (0x195, 0x1195), (0x1a1, 0x1b9),
        (0x1a3, 0x9a3), (0x1a5, 0x1ad), (0x1d1, 0x3d1), (0x213, 0xe13),
        (0x215, 0x1295), (0x21d, 0x31d), (0x229, 0x1269), (0x22b, 0x122b),
        (0x22d, 0xa2d), (0x231, 0x12b1), (0x233, 0x2233), (0x235, 0x1235),
        (0x239, 0x2239), (0x243, 0xa47), (0x24b, 0xa4b), (0x24d, 0x2cd),
        (0x253, 0x2d3), (0x255, 0x1255), (0x271, 0xa71), (0x2c5, 0x12c5),
        (0x2c9, 0x3c9), (0x30d, 0x32d), (0x313, 0x353), (0x389, 0x3a9),
        (0x445, 0x2c45), (0x447, 0x1447), (0x455, 0x4d5), (0x469, 0x479),
        (0x493, 0x4d3), (0x495, 0x49d), (0x4a3, 0x5a3), (0x4c9, 0xcc9),
        (0x519, 0x51d), (0x525, 0x725), (0x549, 0x749), (0x649, 0x64b),
    ]),
    (17, 2, 7, [
        (0x3, 0x7f), (0x5, 0xfd), (0x9, 0x1f9), (0x11, 0x3f1),
        (0x21, 0x7e1), (0x41, 0xfc1), (0x81, 0xf83), (0x87, 0xbf),
        (0x8b, 0xfb), (0x93, 0x1f3), (0xa3, 0x3e3), (0xc3, 0x3c7),
        (0xcf, 0xdf), (0xd7, 0xf7), (0xe7, 0xef), (0x101, 0xf07),
        (0x10b, 0x13f), (0x10d, 0x17d), (0x113, 0x177), (0x115, 0x1f5),
        (0x123, 0x367), (0x125, 0x3e5), (0x143, 0x34f), (0x145, 0x7c5),
        (0x15b, 0x15f), (0x16b, 0x16f), (0x185, 0x38f), (0x18b, 0x1bb),
        (0x197, 0x19f), (0x19d, 0x1bd), (0x1a7, 0x1af), (0x1ad, 0x1ed),
        (0x1cb, 0x1cf), (0x1cd, 0x1dd), (0x213, 0x23f), (0x215, 0x27d),
        (0x219, 0x2f9), (0x223, 0x26f), (0x225, 0x2ed), (0x229, 0x3e9),
        (0x243, 0x25f), (0x245, 0x6cd), (0x249, 0x7c9), (0x273, 0x277),
        (0x285, 0x29f), (0x289, 0x78b), (0x293, 0x2bb), (0x2a7, 0x2af),
        (0x2ab, 0x2eb), (0x2b5, 0x2b7), (0x2cb, 0x2cf), (0x2d3, 0x2d7),
        (0x2d5, 0x2dd), (0x309, 0x31f), (0x313, 0x337), (0x315, 0x375),
        (0x32b, 0x32f), (0x32d, 0x33d), (0x339, 0x33b), (0x34d, 0x35d),
        (0x353, 0x357), (0x359, 0x35b), (0x393, 0x397), (0x395, 0x39d),
        (0x399, 0x39b), (0x423, 0x43f), (0x425, 0x47d), (0x429, 0x4f9),
        (0x431, 0x5f1), (0x443, 0x45f), (0x445, 0x4dd), (0x449, 0x5d9),
        (0x451, 0x7d1), (0x463, 0x46f), (0x473, 0x477), (0x485, 0x49f),
        (0x489, 0x59b), (0x491, 0x793), (0x4a3, 0x4af), (0x4a5, 0x4bd),
        (0x4b3, 0x4b7), (0x4c3, 0x4cf), (0x4d3, 0x4d7), (0x4e3, 0x4e7),
        (0x4e5, 0x4ed), (0x509, 0x51f), (0x511, 0x717), (0x523, 0x52f),
        (0x525, 0x53d), (0x529, 0x579), (0x533, 0x537), (0x543, 0x54f),
        (0x54d, 0x55d), (0x553, 0x557), (0x555, 0x575), (0x563, 0x567),
        (0x565, 0x56d), (0x585, 0x58f), (0x595, 0x597), (0x5a3, 0x5a7),
        (0x5a5, 0x5ad), (0x5a9, 0x5ab), (0x5c3, 0x5c7), (0x611, 0x61f),
        (0x623, 0x62f), (0x625, 0x63d), (0x629, 0x679), (0x631, 0x673),
```

```
(0x643, 0x64f), (0x653, 0x657), (0x655, 0x65d), (0x659, 0x65b),
(0x663, 0x667), (0x665, 0x66d), (0x685, 0x68f), (0x695, 0x697),
(0x699, 0x69b), (0x6a3, 0x6a7), (0x6a5, 0x6ad), (0x6a9, 0x6ab),
(0x6b1, 0x6b3), (0x6c3, 0x6c7), (0x709, 0x70f), (0x719, 0x71b),
(0x723, 0x727), (0x725, 0x72d), (0x729, 0x72b), (0x731, 0x733),
(0x743, 0x747), (0x785, 0x787), (0x843, 0x85f), (0x845, 0x87d),
(0x849, 0x8f9), (0x851, 0x9f1), (0x861, 0x9e3), (0x867, 0x86f),
(0x86b, 0x87b), (0x873, 0x877), (0x885, 0x18b5), (0x887, 0x8b7),
(0x889, 0x89f), (0x891, 0x9b3), (0x8a1, 0x9ab), (0x8ad, 0x8af),
(0x8b9, 0x8bb), (0x8c3, 0x8cf), (0x8c5, 0x8dd), (0x8d3, 0x8d7),
(0x8e5, 0x8e7), (0x909, 0x91f), (0x911, 0x937), (0x921, 0x92f),
(0x939, 0x93b), (0x943, 0x94f), (0x945, 0x95d), (0x949, 0x979),
(0x953, 0x957), (0x965, 0x967), (0x985, 0x98f), (0x989, 0x99b),
(0x995, 0x997), (0x9a5, 0x9a7), (0x9c3, 0x9c7), (0x9c5, 0x9cd),
(0x9c9, 0x9cb), (0xa11, 0xa1f), (0xa21, 0xa2f), (0xa31, 0xa37),
(0xa39, 0xa3b), (0xa43, 0xa4f), (0xa45, 0xa5d), (0xa49, 0xa79),
(0xa51, 0xa73), (0xa61, 0xa67), (0xa85, 0xa8f), (0xa89, 0xa9b),
(0xa91, 0xa97), (0xaa1, 0xaa7), (0xaa9, 0xaab), (0xab1, 0xab3),
(0xac3, 0xac7), (0xac5, 0xacd), (0xac9, 0xacb), (0xad1, 0xad3),
(0xae1, 0xae3), (0xb09, 0xb0f), (0xb11, 0xb17), (0xb19, 0xb1b),
(0xb21, 0xb27), (0xb29, 0xb2b), (0xb31, 0xb33), (0xb43, 0xb47),
(0xb45, 0xb4d), (0xb49, 0xb4b), (0xb51, 0xb53), (0xb61, 0xb63),
(0xb85, 0xb87), (0xb89, 0xb8b), (0xb91, 0xb93), (0xba1, 0xba3),
(0xc21, 0xc2f), (0xc31, 0xc37), (0xc39, 0xc3b), (0xc43, 0xc4f),
(0xc45, 0xc5d), (0xc49, 0xc79), (0xc51, 0xc73), (0xc61, 0xc67),
(0xc85, 0xc8f), (0xc89, 0xc9b), (0xc91, 0xc97), (0xca1, 0xca7),
(0xca9, 0xcab), (0xcb1, 0xcb3), (0xcc3, 0xcc7), (0xcc5, 0xccd),
(0xcc9, 0xccb), (0xcd1, 0xcd3), (0xce1, 0xce3), (0xd09, 0xd0f),
(0xd11, 0xd17), (0xd19, 0xd1b), (0xd21, 0xd27), (0xd29, 0xd2b),
(0xd31, 0xd33), (0xd43, 0xd47), (0xd45, 0xd4d), (0xd49, 0xd4b),
(0xd51, 0xd53), (0xd61, 0xd63), (0xd85, 0xd87), (0xd89, 0xd8b),
(0xd91, 0xd93), (0xda1, 0xda3), (0xe11, 0xe17), (0xe19, 0xe1b),
(0xe21, 0xe27), (0xe29, 0xe2b), (0xe31, 0xe33), (0xe43, 0xe47),
(0xe45, 0xe4d), (0xe49, 0xe4b), (0xe51, 0xe53), (0xe61, 0xe63),
(0xe85, 0xe87), (0xe89, 0xe8b), (0xe91, 0xe93), (0xea1, 0xea3),
(0xf09, 0xf0b), (0xf11, 0xf13), (0xf21, 0xf23), (0x1085, 0x109f),
(0x1089, 0x10bb), (0x1091, 0x10f3), (0x10a3, 0x10e7), (0x10a5, 0x10bd),
(0x10c3, 0x10cf), (0x10c5, 0x10dd), (0x10c9, 0x10f9), (0x10e5, 0x10ed),
(0x1109, 0x111f), (0x1111, 0x1137), (0x1123, 0x112f), (0x1125, 0x116d),
(0x1129, 0x1179), (0x1143, 0x114f), (0x1145, 0x115d), (0x1149, 0x11d9),
(0x1151, 0x1173), (0x1163, 0x1167), (0x1185, 0x118f), (0x1189, 0x119b),
(0x1191, 0x1197), (0x11a3, 0x11a7), (0x11a5, 0x11ad), (0x11a9, 0x11ab),
(0x11b1, 0x11b3), (0x11c3, 0x11c7), (0x11c5, 0x11cd), (0x11d1, 0x11d3),
(0x1213, 0x121f), (0x1215, 0x123d), (0x1219, 0x1279), (0x1223, 0x122f),
(0x1225, 0x126d), (0x1229, 0x12e9), (0x1231, 0x1273), (0x1243, 0x124f),
(0x1245, 0x125d), (0x1249, 0x13c9), (0x1251, 0x12d3), (0x1263, 0x1267),
(0x1285, 0x128f), (0x1289, 0x129b), (0x1291, 0x1297), (0x12a3, 0x12a7),
(0x12a5, 0x12ad), (0x12b1, 0x12b3), (0x12c3, 0x12c7), (0x12c5, 0x12cd),
(0x130b, 0x130f), (0x130d, 0x131d), (0x1311, 0x1317), (0x1319, 0x131b),
(0x1323, 0x1327), (0x1325, 0x132d), (0x1329, 0x132b), (0x1331, 0x1333),
(0x1343, 0x1347), (0x1345, 0x134d), (0x1351, 0x1353), (0x1385, 0x1387),
(0x1389, 0x138b), (0x1391, 0x1393), (0x1429, 0x142f), (0x1433, 0x1437),
(0x1435, 0x143d), (0x1439, 0x143b), (0x1443, 0x144f), (0x1445, 0x145d),
(0x1449, 0x1479), (0x1451, 0x1473), (0x1463, 0x1467), (0x1465, 0x146d),
(0x1487, 0x14a7), (0x1489, 0x148f), (0x1491, 0x1497), (0x1499, 0x149b),
(0x14a3, 0x14ab), (0x14a5, 0x14ad), (0x14a9, 0x14b9), (0x14b1, 0x14b3),
(0x14c3, 0x14c7), (0x14c5, 0x14cd), (0x14c9, 0x14cb), (0x14d1, 0x14d3),
(0x150b, 0x150f), (0x150d, 0x151d), (0x1511, 0x1517), (0x1519, 0x151b),
(0x1523, 0x1527), (0x1525, 0x152d), (0x1529, 0x152b), (0x1531, 0x1533),
(0x1543, 0x1547), (0x1545, 0x154d), (0x1549, 0x154b), (0x1551, 0x1553),
(0x1589, 0x158b), (0x1591, 0x1593), (0x1619, 0x161b), (0x1623, 0x1627),
(0x1625, 0x162d), (0x1629, 0x162b), (0x1631, 0x1633), (0x1643, 0x1647),
(0x1645, 0x164d), (0x1649, 0x164b), (0x1651, 0x1653), (0x1689, 0x168b),
(0x1691, 0x1693), (0x1711, 0x1713), (0x1863, 0x1867), (0x1865, 0x186d),
(0x1869, 0x186b), (0x1871, 0x1873), (0x1887, 0x1897), (0x1889, 0x188f),
(0x1891, 0x189b), (0x18a3, 0x18a7), (0x18a9, 0x18ab), (0x18b1, 0x18b3),
(0x18c5, 0x18c7), (0x18c9, 0x18cb), (0x18d1, 0x18d3), (0x190d, 0x190f),
(0x1911, 0x1917), (0x1919, 0x191b), (0x1923, 0x1927), (0x1925, 0x192d),
(0x1929, 0x192b), (0x1931, 0x1933), (0x1945, 0x1947), (0x1949, 0x194b),
(0x1951, 0x1953), (0x1989, 0x198b), (0x1991, 0x1993), (0x1a23, 0x1a27),
(0x1a25, 0x1a2d), (0x1a29, 0x1a2b), (0x1a31, 0x1a33), (0x1a45, 0x1a47),
(0x1a49, 0x1a4b), (0x1a51, 0x1a53), (0x1a89, 0x1a8b), (0x1a91, 0x1a93),
(0x1b11, 0x1b13), (0x1c45, 0x1c47), (0x1c49, 0x1c4b), (0x1c51, 0x1c53),
```

```
    (0x1c89, 0x1c8b), (0x1c91, 0x1c93), (0x1d11, 0x1d13), (0x2223, 0x222f),
    (0x2225, 0x223d), (0x2229, 0x2279), (0x2233, 0x2237), (0x2245, 0x224f),
    (0x2249, 0x225b), (0x2253, 0x2257), (0x2255, 0x225d), (0x2263, 0x2267),
    (0x2265, 0x226d), (0x2289, 0x6299), (0x228b, 0x228f), (0x228d, 0x229d),
    (0x2293, 0x2297), (0x2295, 0x22b5), (0x22a3, 0x22a7), (0x22a5, 0x22ad),
    (0x22a9, 0x22ab), (0x22c5, 0x22c7), (0x22c9, 0x22cb), (0x2313, 0x2317),
    (0x2315, 0x231d), (0x2319, 0x231b), (0x2323, 0x2333), (0x2325, 0x2327),
    (0x2329, 0x232b), (0x2345, 0x2347), (0x2349, 0x234b), (0x2389, 0x238b),
    (0x2449, 0x244f), (0x2453, 0x2457), (0x2455, 0x245d), (0x2459, 0x245b),
    (0x2463, 0x2467), (0x2465, 0x246d), (0x2469, 0x246b), (0x248b, 0xa48b),
    (0x248d, 0x248f), (0x2493, 0x2497), (0x2495, 0x249d), (0x2499, 0x249b),
    (0x24a3, 0x24a7), (0x24a5, 0x24ad), (0x24a9, 0x24ab), (0x24c9, 0x24cb),
    (0x2513, 0x2517), (0x2515, 0x251d), (0x2519, 0x251b), (0x2523, 0x2527),
    (0x2525, 0x252d), (0x2529, 0x252b), (0x2545, 0x2547), (0x2549, 0x254b),
    (0x2629, 0x262b), (0x2645, 0x2647), (0x2649, 0x264b), (0x28a3, 0x28a7),
    (0x28a5, 0x28ad), (0x28a9, 0x28ab), (0x28c9, 0x28cb), (0x2915, 0x2917),
    (0x2919, 0x291b), (0x2923, 0x2927), (0x2925, 0x292d), (0x2929, 0x2939),
    (0x2949, 0x294b), (0x2a49, 0x2a4b), (0x3249, 0x324b), (0x4925, 0x14925),
]),
(19, 3, 7, [
    (0x7, 0x7f), (0xb, 0xfb), (0xd, 0xfd), (0x13, 0x1f3),
    (0x15, 0x1f5), (0x19, 0x1f9), (0x23, 0x3e3), (0x25, 0xe65),
    (0x29, 0x3e9), (0x31, 0x3f1), (0x43, 0x7c3), (0x45, 0x7c5),
    (0x49, 0x7c9), (0x51, 0x7d1), (0x61, 0x7e1), (0x83, 0x787),
    (0x85, 0x7a5), (0x89, 0xf89), (0x8f, 0xbf), (0x91, 0xf91),
    (0x97, 0xf7), (0xa1, 0xfa1), (0xa7, 0x1e7), (0xc1, 0xfc1),
    (0xc7, 0x1cf), (0xe5, 0x3e5), (0x103, 0x70f), (0x105, 0x72d),
    (0x109, 0x1f09), (0x111, 0x1f11), (0x117, 0x13f), (0x11b, 0x17b),
    (0x11d, 0x17d), (0x121, 0x1f21), (0x127, 0x16f), (0x12b, 0x1eb),
    (0x141, 0x1f41), (0x147, 0x15f), (0x14b, 0x3cb), (0x14d, 0x1ed),
    (0x165, 0x765), (0x181, 0x1f81), (0x18b, 0x19f), (0x18d, 0x1bd),
    (0x197, 0x1b7), (0x203, 0x61f), (0x205, 0xe1d), (0x209, 0x1e19),
    (0x211, 0x3e11), (0x221, 0x3e21), (0x227, 0x23f), (0x22b, 0x27b),
    (0x22d, 0x27d), (0x233, 0x2f3), (0x235, 0x2f5), (0x239, 0x2f9),
    (0x241, 0x3e41), (0x247, 0x25f), (0x24b, 0x2db), (0x24d, 0x2dd),
    (0x253, 0x3d3), (0x255, 0x3d5), (0x259, 0x3d9), (0x267, 0x26f),
    (0x281, 0x3e81), (0x28b, 0x29f), (0x28d, 0x2bd), (0x293, 0x397),
    (0x295, 0x795), (0x299, 0x799), (0x2a7, 0x2af), (0x2ab, 0x2bb),
    (0x2c7, 0x2cf), (0x301, 0x7701), (0x313, 0x31f), (0x315, 0x33d),
    (0x319, 0x739), (0x327, 0x32f), (0x32b, 0x33b), (0x333, 0x337),
    (0x347, 0x34f), (0x34d, 0x35d), (0x38b, 0x38f), (0x38d, 0x39d),
    (0x403, 0x43f), (0x405, 0x40c1d), (0x409, 0x1c39), (0x411, 0x3c31),
    (0x421, 0x7c21), (0x42d, 0x47d), (0x435, 0x4f5), (0x441, 0x7c41),
    (0x447, 0x45f), (0x44b, 0x47b), (0x44d, 0x4dd), (0x453, 0x4f3),
    (0x455, 0x5d5), (0x459, 0x4f9), (0x463, 0x4e7), (0x469, 0x5e9),
    (0x471, 0x5f1), (0x481, 0x7c81), (0x48b, 0x49f), (0x48d, 0x4bd),
    (0x493, 0x4b7), (0x495, 0x1595), (0x499, 0x5b9), (0x4a3, 0x4af),
    (0x4a9, 0x7a9), (0x4b1, 0x7b1), (0x4c7, 0x4cf), (0x4cb, 0x4db),
    (0x4e5, 0x4ed), (0x501, 0xdd01), (0x513, 0x51f), (0x515, 0x53d),
    (0x519, 0x579), (0x523, 0x52f), (0x529, 0xf29), (0x531, 0x733),
    (0x547, 0x54f), (0x54b, 0x55b), (0x54d, 0x55d), (0x553, 0x557),
    (0x563, 0x567), (0x58b, 0x58f), (0x58d, 0x59d), (0x593, 0x597),
    (0x5a3, 0x5a7), (0x623, 0x62f), (0x629, 0x63d), (0x631, 0x637),
    (0x647, 0x64f), (0x64b, 0x65b), (0x64d, 0x65d), (0x653, 0x657),
    (0x655, 0x675), (0x659, 0x679), (0x663, 0x667), (0x669, 0x66b),
    (0x671, 0x673), (0x68b, 0x68f), (0x68d, 0x69d), (0x693, 0x697),
    (0x6a3, 0x6a7), (0x713, 0x717), (0x715, 0x71d), (0x723, 0x727),
    (0x809, 0x3839), (0x80f, 0x83f), (0x811, 0x3871), (0x821, 0x16861),
    (0x82d, 0x87d), (0x835, 0x8f5), (0x841, 0x43849), (0x84b, 0x85f),
    (0x859, 0x8f9), (0x869, 0x8eb), (0x881, 0x7883), (0x887, 0x89f),
    (0x88b, 0x8bb), (0x893, 0x8f3), (0x899, 0x9b9), (0x8a3, 0x8e7),
    (0x8a5, 0x8ed), (0x8a9, 0xba9), (0x8b1, 0x9f1), (0x8c3, 0x8cf),
    (0x8c9, 0xbc9), (0x8d1, 0xbd1), (0x8e1, 0xbe1), (0x90b, 0x91f),
    (0x90d, 0x93d), (0x913, 0x937), (0x915, 0x975), (0x919, 0x979),
    (0x923, 0x92f), (0x925, 0x9e5), (0x929, 0xb69), (0x931, 0xb71),
    (0x943, 0x94f), (0x945, 0x9cd), (0x949, 0xf49), (0x951, 0xb53),
    (0x961, 0xb63), (0x983, 0x20d83), (0x985, 0xd95), (0x987, 0x98f),
    (0x98b, 0x99b), (0x98d, 0x99d), (0x993, 0x997), (0x9a3, 0x9a7),
    (0x9c3, 0x9c7), (0xa07, 0xa1f), (0xa0b, 0xa3b), (0xa13, 0xa73),
    (0xa23, 0xa67), (0xa29, 0xa3d), (0xa31, 0xa75), (0xa43, 0xa4f),
    (0xa45, 0xa5d), (0xa49, 0xa79), (0xa51, 0x1e51), (0xa61, 0x1e61),
    (0xa83, 0xa8f), (0xa85, 0xa9d), (0xa93, 0xa97), (0xa99, 0xa9b),
    (0xaa3, 0xaa7), (0xaa5, 0xaad), (0xab1, 0xab3), (0xac3, 0xac7),
```

```
(0xac5, 0xacd), (0xb05, 0xb0f), (0xb0b, 0xb1b), (0xb13, 0xb17),
(0xb15, 0xb1d), (0xb19, 0xb39), (0xb23, 0xb27), (0xb25, 0xb2d),
(0xb43, 0xb47), (0xb45, 0xb4d), (0xb83, 0xb87), (0xb85, 0xb8d),
(0xc07, 0xc1f), (0xc0b, 0xc3b), (0xc13, 0xc73), (0xc23, 0xc67),
(0xc2d, 0xc2f), (0xc35, 0xc37), (0xc43, 0xc4f), (0xc45, 0xc5d),
(0xc49, 0xc79), (0xc51, 0xcf1), (0xc61, 0xde1), (0xc83, 0xc8f),
(0xc85, 0xc9d), (0xc93, 0xc97), (0xc99, 0xc9b), (0xca3, 0xca7),
(0xca5, 0xcad), (0xca9, 0xcab), (0xcb1, 0xcb3), (0xcc3, 0xcc7),
(0xcc5, 0xccd), (0xcc9, 0xccb), (0xd03, 0xd0f), (0xd05, 0xd1d),
(0xd13, 0xd17), (0xd19, 0xd1b), (0xd23, 0xd27), (0xd25, 0xd2d),
(0xd31, 0xd33), (0xd43, 0xd47), (0xd45, 0xd4d), (0xd51, 0xd53),
(0xe07, 0xe0f), (0xe0b, 0xe1b), (0xe13, 0xe17), (0xe23, 0xe27),
(0xe29, 0xe2b), (0xe31, 0xe33), (0xe43, 0xe47), (0xe45, 0xe4d),
(0xe49, 0xe4b), (0xe83, 0xe87), (0xe85, 0xe8d), (0xf03, 0xf07),
(0xf05, 0xf15), (0x1021, 0x58e1), (0x1041, 0x450c3), (0x1047, 0x105f),
(0x104b, 0x107b), (0x1053, 0x10f3), (0x1063, 0x10e7), (0x1087, 0x5187),
(0x108b, 0x109f), (0x108d, 0x10bd), (0x1095, 0x10b7), (0x10a3, 0x10af),
(0x10a5, 0x11e5), (0x10b3, 0x10bb), (0x10c5, 0x10cf), (0x10cb, 0x10db),
(0x10d5, 0x10d7), (0x110b, 0x111f), (0x110d, 0x113d), (0x1113, 0x1137),
(0x1115, 0x1175), (0x1119, 0x1179), (0x1123, 0x112f), (0x1125, 0x1365),
(0x1129, 0x11e9), (0x1131, 0x11f1), (0x1143, 0x114f), (0x1145, 0x11cd),
(0x1149, 0x13c9), (0x1151, 0x11d3), (0x1161, 0x11e3), (0x1183, 0x119b),
(0x1185, 0x119d), (0x1189, 0x1399), (0x1191, 0x1791), (0x11a1, 0x13a3),
(0x11c1, 0x13c3), (0x120b, 0x121f), (0x120d, 0x123d), (0x1213, 0x1237),
(0x1215, 0x1275), (0x1223, 0x122f), (0x1225, 0x12e5), (0x1229, 0x1279),
(0x1231, 0x12f1), (0x1243, 0x124f), (0x1245, 0x12cd), (0x1249, 0x12d9),
(0x1251, 0x1353), (0x1261, 0x12e3), (0x1283, 0x128f), (0x1285, 0x21385),
(0x1289, 0x328d), (0x1291, 0x1699), (0x1293, 0x1297), (0x1295, 0x129d),
(0x12a1, 0x216a1), (0x12a3, 0x12a7), (0x12a9, 0x12ab), (0x12c1, 0x12d3),
(0x1303, 0x130f), (0x1305, 0x131d), (0x1313, 0x1317), (0x1319, 0x131b),
(0x1323, 0x1327), (0x1329, 0x132b), (0x1331, 0x1333), (0x1343, 0x1347),
(0x1345, 0x134d), (0x1361, 0x1363), (0x1383, 0x1387), (0x140b, 0x41c0b),
(0x140d, 0x141f), (0x1413, 0x1437), (0x1415, 0x1535), (0x141b, 0x143b),
(0x1423, 0x142f), (0x1425, 0x146d), (0x1443, 0x144f), (0x1445, 0x14cd),
(0x1449, 0x1479), (0x1451, 0x1473), (0x1455, 0x21455), (0x1461, 0x14e3),
(0x1483, 0x148f), (0x1485, 0x14ad), (0x1489, 0x14b9), (0x1491, 0x14b3),
(0x14a1, 0x15a3), (0x14c1, 0x14cb), (0x14d1, 0x14d3), (0x1503, 0x150f),
(0x1505, 0x152d), (0x1513, 0x1517), (0x1519, 0x151b), (0x1523, 0x1527),
(0x1529, 0x152b), (0x1531, 0x1533), (0x1543, 0x1547), (0x1545, 0x154d),
(0x1549, 0x154b), (0x1551, 0x1553), (0x1561, 0x1563), (0x1583, 0x1587),
(0x1585, 0x158d), (0x1589, 0x158b), (0x15c1, 0x15c3), (0x160b, 0x160f),
(0x160d, 0x161d), (0x1613, 0x1617), (0x1615, 0x1635), (0x1623, 0x1627),
(0x1625, 0x162d), (0x1629, 0x162b), (0x1631, 0x1633), (0x1643, 0x1647),
(0x1645, 0x164d), (0x1649, 0x164b), (0x1651, 0x1653), (0x1661, 0x1663),
(0x1683, 0x1687), (0x1685, 0x168d), (0x1689, 0x168b), (0x16c1, 0x16c3),
(0x1703, 0x170b), (0x181d, 0x181f), (0x1823, 0x1837), (0x1825, 0x183d),
(0x1843, 0x184f), (0x1845, 0x185d), (0x1853, 0x1857), (0x1859, 0x185b),
(0x1863, 0x1867), (0x1865, 0x186d), (0x1869, 0x186b), (0x1885, 0x188f),
(0x1889, 0x18ab), (0x1891, 0x18b3), (0x1895, 0x5895), (0x1899, 0x189d),
(0x18a3, 0x18a7), (0x18a5, 0x18ad), (0x18c1, 0x18d3), (0x18c5, 0x1ac5),
(0x18c9, 0x18d9), (0x190b, 0x190f), (0x190d, 0x191d), (0x1913, 0x1917),
(0x1915, 0x1935), (0x1919, 0x191b), (0x1923, 0x9923), (0x1925, 0x9925),
(0x1929, 0x9929), (0x1931, 0x1933), (0x1943, 0x3943), (0x1945, 0x1947),
(0x1949, 0x194b), (0x1951, 0x1953), (0x1961, 0x1963), (0x1983, 0x1987),
(0x1985, 0x198d), (0x1989, 0x198b), (0x1991, 0x1993), (0x19a1, 0x19a3),
(0x19c1, 0x19c3), (0x1a07, 0x1a0f), (0x1a0b, 0x1a1b), (0x1a0d, 0x1a1d),
(0x1a13, 0x1a17), (0x1a15, 0x1a35), (0x1a23, 0x1a27), (0x1a25, 0x1a2d),
(0x1a29, 0x1a2b), (0x1a31, 0x1a33), (0x1a43, 0x1a47), (0x1a45, 0x1a4d),
(0x1a49, 0x1a4b), (0x1a83, 0x1a87), (0x1a85, 0x1a8d), (0x1a89, 0x1a8b),
(0x1a91, 0x1a93), (0x1aa1, 0x1aa3), (0x1ac1, 0x1ac3), (0x1b05, 0x1b07),
(0x1c13, 0x1c17), (0x1c15, 0x1c1d), (0x1c23, 0x1c27), (0x1c25, 0x1c2d),
(0x1c43, 0x1c47), (0x1c45, 0x1c4d), (0x1c49, 0x1c4b), (0x1c51, 0x1c53),
(0x1c61, 0x1c63), (0x1c83, 0x1c87), (0x1c85, 0x1c8d), (0x1c89, 0x1c8b),
(0x1c91, 0x1c93), (0x1ca1, 0x1ca3), (0x1cc1, 0x1cc3), (0x1d05, 0x1d07),
(0x208b, 0x209f), (0x208d, 0x20bd), (0x2093, 0x20b7), (0x2095, 0x20f5),
(0x20a3, 0x20af), (0x20a5, 0x21e5), (0x20c3, 0x20cf), (0x20cd, 0x20dd),
(0x20d3, 0x20d7), (0x20e3, 0x20e7), (0x2107, 0x211f), (0x2113, 0x213b),
(0x2115, 0x213d), (0x2119, 0x2179), (0x2123, 0x212f), (0x2145, 0x214f),
(0x2149, 0x216b), (0x2153, 0x2157), (0x2155, 0x215d), (0x2161, 0x2167),
(0x2189, 0x218f), (0x2193, 0x2197), (0x2195, 0x219d), (0x2199, 0x219b),
(0x21a1, 0x21ab), (0x21c3, 0x21c7), (0x21c9, 0x21cb), (0x21e1, 0x21e3),
(0x2213, 0x221f), (0x2215, 0x223d), (0x2219, 0x2279), (0x2223, 0x222f),
(0x2225, 0x226d), (0x2229, 0x22e9), (0x2231, 0x2273), (0x2243, 0x224f),
```

```
(0x2245, 0x225d), (0x2249, 0x23c9), (0x2251, 0x22d3), (0x2261, 0x22e3),
(0x2283, 0x228f), (0x2285, 0x229d), (0x2289, 0x2399), (0x2291, 0x22b3),
(0x22a1, 0x22a7), (0x22c3, 0x22c7), (0x22c5, 0x22cd), (0x2305, 0x230f),
(0x2309, 0x231b), (0x2311, 0x2317), (0x2321, 0x2327), (0x2329, 0x232b),
(0x2331, 0x2333), (0x2343, 0x2347), (0x2345, 0x234d), (0x2351, 0x2353),
(0x2361, 0x2363), (0x2383, 0x2387), (0x2385, 0x238d), (0x2391, 0x2393),
(0x23a1, 0x23a3), (0x2413, 0x241f), (0x2415, 0x243d), (0x2419, 0x2479),
(0x2423, 0x242f), (0x2425, 0x246d), (0x2429, 0x24e9), (0x2433, 0x2437),
(0x2443, 0x244f), (0x2445, 0x245d), (0x2449, 0x25c9), (0x2451, 0x2473),
(0x2461, 0x2563), (0x2483, 0x248f), (0x2485, 0x249d), (0x2489, 0x2789),
(0x2491, 0x2c99), (0x2493, 0x2497), (0x24a1, 0x124e1), (0x24a3, 0x24a7),
(0x24a5, 0x24ad), (0x24b1, 0x24b3), (0x24c3, 0x24c7), (0x24c5, 0x24cd),
(0x24d1, 0x24d3), (0x2505, 0x250f), (0x2509, 0x251b), (0x2511, 0x2517),
(0x2521, 0x6523), (0x2525, 0x2527), (0x2529, 0x252b), (0x2531, 0x2533),
(0x2543, 0x2547), (0x2545, 0x254d), (0x2551, 0x2553), (0x2583, 0x2587),
(0x2585, 0x258d), (0x2591, 0x2593), (0x25a1, 0x25a3), (0x260b, 0x2260b),
(0x2613, 0x2617), (0x2615, 0x261d), (0x2619, 0x261b), (0x2623, 0x2627),
(0x2625, 0x262d), (0x2629, 0x262b), (0x2631, 0x2633), (0x2643, 0x2663),
(0x2645, 0x2647), (0x2649, 0x6649), (0x2651, 0x2653), (0x2661, 0x2665),
(0x2683, 0x2687), (0x2685, 0x268d), (0x2691, 0x2693), (0x26a1, 0x26a3),
(0x2705, 0x2707), (0x2709, 0x270b), (0x2711, 0x2713), (0x2721, 0x2723),
(0x282b, 0x282f), (0x282d, 0x283d), (0x2833, 0x2837), (0x2835, 0x2875),
(0x2843, 0x284f), (0x2845, 0x285d), (0x2853, 0x2857), (0x2859, 0x285b),
(0x2863, 0x2867), (0x2865, 0x286d), (0x2869, 0x286b), (0x2885, 0x288f),
(0x2889, 0x289b), (0x2891, 0x2897), (0x28a1, 0x28a7), (0x28a9, 0x28ab),
(0x28b1, 0x28b3), (0x28c3, 0x28c7), (0x28c5, 0x28cd), (0x28c9, 0x28cb),
(0x28d1, 0x28d3), (0x28e1, 0x28e3), (0x2909, 0x290f), (0x2911, 0x2917),
(0x2919, 0x291b), (0x2921, 0x2927), (0x2929, 0x292b), (0x2931, 0x2933),
(0x2943, 0x2947), (0x2945, 0x294d), (0x2949, 0x294b), (0x2951, 0x2953),
(0x2961, 0x2963), (0x2983, 0x2987), (0x2985, 0x298d), (0x2989, 0x298b),
(0x2991, 0x2993), (0x29a1, 0x29a3), (0x2a0b, 0x12a0b), (0x2a0d, 0x2a4d),
(0x2a13, 0x2a53), (0x2a15, 0x2a17), (0x2a19, 0x2a1b), (0x2a23, 0x2a27),
(0x2a25, 0x2a2d), (0x2a29, 0x2a2b), (0x2a31, 0x2a33), (0x2a43, 0x2a47),
(0x2a45, 0x2a55), (0x2a49, 0x2a4b), (0x2a51, 0x2a59), (0x2a61, 0x2a63),
(0x2a83, 0x2a87), (0x2a85, 0x2a8d), (0x2a89, 0x2a8b), (0x2a91, 0x2a93),
(0x2aa1, 0x2aa3), (0x2b09, 0x2b0b), (0x2b11, 0x2b13), (0x2b21, 0x2b23),
(0x2c19, 0x2c1b), (0x2c23, 0x2c27), (0x2c25, 0x2c2d), (0x2c29, 0x2c2b),
(0x2c43, 0x2c47), (0x2c45, 0x2c4d), (0x2c49, 0x2c4b), (0x2c51, 0x2c53),
(0x2c61, 0x2c63), (0x2c83, 0x2c87), (0x2c85, 0x2c8d), (0x2c89, 0x2c8b),
(0x2ca1, 0x2ca3), (0x2d09, 0x2d0b), (0x2d11, 0x2d15), (0x2d21, 0x2d23),
(0x3063, 0x3067), (0x3065, 0x306d), (0x3069, 0x306b), (0x3087, 0x30a7),
(0x3089, 0x308f), (0x3095, 0x3295), (0x3099, 0x309b), (0x30a3, 0x30ab),
(0x30a5, 0x30ad), (0x30a9, 0x70a9), (0x30c3, 0x38c3), (0x30c5, 0x30c7),
(0x30c9, 0x30cb), (0x3109, 0x310f), (0x3111, 0x3117), (0x3119, 0x311b),
(0x3121, 0x3127), (0x3129, 0x312b), (0x3131, 0x3133), (0x3145, 0x3147),
(0x3149, 0x314b), (0x3151, 0x3153), (0x3161, 0x3163), (0x3185, 0x3187),
(0x3189, 0x318b), (0x3191, 0x3193), (0x31a1, 0x31a3), (0x320d, 0x320f),
(0x3213, 0x3217), (0x3215, 0x321d), (0x3219, 0x321b), (0x3223, 0x3227),
(0x3225, 0x322d), (0x3229, 0x322b), (0x3231, 0x3233), (0x3243, 0x3247),
(0x3245, 0x324d), (0x3249, 0x324b), (0x3251, 0x3253), (0x3261, 0x3263),
(0x3285, 0x3287), (0x3291, 0x3293), (0x32a1, 0x32a3), (0x3309, 0x330b),
(0x3311, 0x3313), (0x3321, 0x3323), (0x3423, 0x3427), (0x3425, 0x342d),
(0x3429, 0x342b), (0x3443, 0x3447), (0x3445, 0x344d), (0x3449, 0x344b),
(0x3451, 0x3453), (0x3461, 0x3463), (0x3485, 0x3487), (0x3489, 0x348b),
(0x3491, 0x3493), (0x34a1, 0x34a3), (0x3509, 0x350b), (0x3511, 0x3513),
(0x3521, 0x3523), (0x3843, 0x3847), (0x3845, 0x384d), (0x3861, 0x3863),
(0x3885, 0x3887), (0x3889, 0x388b), (0x3891, 0x3893), (0x38a1, 0x38a3),
(0x3909, 0x390b), (0x3911, 0x3913), (0x3921, 0x3923), (0x4223, 0x4237),
(0x4225, 0x1c225), (0x4229, 0x4a2b), (0x422d, 0x422f), (0x4231, 0x423d),
(0x4245, 0x424f), (0x4249, 0x425b), (0x4251, 0x4257), (0x4263, 0x4267),
(0x4265, 0x426d), (0x4269, 0x426b), (0x4271, 0x4273), (0x4289, 0x428f),
(0x4291, 0x4297), (0x4299, 0x429b), (0x42a3, 0x42a7), (0x42a5, 0x42ad),
(0x42a9, 0x42ab), (0x42b1, 0x42b3), (0x42c9, 0x42cb), (0x42d1, 0x42d3),
(0x430b, 0x430f), (0x430d, 0x431d), (0x4311, 0x4317), (0x4319, 0x14319),
(0x4323, 0x432b), (0x4325, 0x4327), (0x4331, 0x4333), (0x4345, 0x4355),
(0x4349, 0x434b), (0x4351, 0x4353), (0x4389, 0x438b), (0x4391, 0x4393),
(0x4423, 0x442f), (0x4425, 0x443d), (0x4429, 0x4479), (0x4431, 0x4473),
(0x4443, 0x444f), (0x4445, 0x445d), (0x4449, 0x44d9), (0x4451, 0x44d3),
(0x4463, 0x4467), (0x4465, 0x446d), (0x4485, 0x448f), (0x4489, 0x449b),
(0x4491, 0x4497), (0x44a9, 0x44ab), (0x44b1, 0x45b1), (0x44c3, 0x44c7),
(0x44c5, 0x44cd), (0x4509, 0x450f), (0x4511, 0x4517), (0x4519, 0x451b),
(0x4523, 0x4527), (0x4525, 0x452d), (0x4529, 0x452b), (0x4531, 0x4533),
(0x4543, 0x4547), (0x4545, 0x454d), (0x4549, 0x454b), (0x4551, 0x4553),
```

```
        (0x4585, 0x4587), (0x4589, 0x458b), (0x4591, 0x4593), (0x4613, 0x4617),
        (0x4615, 0x44615), (0x4619, 0x461b), (0x4623, 0xc623), (0x4625, 0x462d),
        (0x4629, 0x462b), (0x4631, 0x14631), (0x4643, 0x4663), (0x4645, 0x4647),
        (0x4649, 0x464b), (0x4651, 0x4653), (0x4685, 0x4687), (0x4689, 0x468b),
        (0x4691, 0x4693), (0x4709, 0x470b), (0x4711, 0x4713), (0x4849, 0x484f),
        (0x4851, 0x15851), (0x4853, 0x4857), (0x4855, 0x485d), (0x4859, 0x485b),
        (0x4863, 0x4867), (0x4865, 0x486d), (0x4869, 0x486b), (0x4871, 0x4873),
        (0x4885, 0x488f), (0x4889, 0x2c889), (0x488b, 0x489b), (0x4891, 0x4c93),
        (0x4895, 0x4897), (0x4899, 0x489d), (0x48a3, 0x48a7), (0x48a5, 0x48ad),
        (0x48a9, 0x48ab), (0x48b1, 0x48b3), (0x48c3, 0x48c7), (0x48c5, 0x48cd),
        (0x48c9, 0x48cb), (0x48d1, 0x48d3), (0x490b, 0x490f), (0x490d, 0x491d),
        (0x4911, 0x4917), (0x4919, 0x491b), (0x4923, 0x492b), (0x4929, 0x492d),
        (0x4931, 0x4933), (0x4945, 0x4947), (0x4949, 0x494b), (0x4951, 0x4953),
        (0x4989, 0x498b), (0x4991, 0x4993), (0x4a13, 0x4a17), (0x4a19, 0x4a1b),
        (0x4a23, 0x4a27), (0x4a25, 0x4a2d), (0x4a31, 0x4a33), (0x4a45, 0x4a47),
        (0x4a49, 0x4b49), (0x4a51, 0x4a53), (0x4a89, 0x4a8d), (0x4a91, 0x4a93),
        (0x4b11, 0x4b13), (0x4c29, 0x4c2b), (0x4c31, 0x4c33), (0x4c43, 0x4c47),
        (0x4c45, 0x4c4d), (0x4c49, 0x4c4b), (0x4c51, 0x4c53), (0x4c85, 0x4c87),
        (0x4c89, 0x4c8b), (0x4d11, 0x4d13), (0x50a3, 0x50a7), (0x50a5, 0x50ad),
        (0x50a9, 0xd0a9), (0x50b1, 0x50b3), (0x50c5, 0x50c7), (0x50c9, 0x50cb),
        (0x50d1, 0x50d3), (0x510d, 0x510f), (0x5111, 0xd511), (0x5113, 0x5117),
        (0x5115, 0x511d), (0x5119, 0x511b), (0x5123, 0x5127), (0x5125, 0x45125),
        (0x5131, 0x5331), (0x5143, 0x5147), (0x5145, 0x514d), (0x5149, 0x514b),
        (0x5151, 0x5153), (0x5189, 0x518b), (0x5191, 0x5193), (0x5215, 0x5217),
        (0x5219, 0x521b), (0x5223, 0x5227), (0x5225, 0x522d), (0x5229, 0x522b),
        (0x5231, 0x5233), (0x5243, 0x5247), (0x5245, 0x524d), (0x5249, 0x524b),
        (0x5251, 0x5253), (0x5289, 0x528b), (0x5291, 0x5293), (0x5311, 0x5313),
        (0x5431, 0x5433), (0x5443, 0x5447), (0x5445, 0x544d), (0x5449, 0x544b),
        (0x5451, 0x5453), (0x5489, 0x548b), (0x5491, 0x5493), (0x5889, 0x588b),
        (0x5891, 0x5899), (0x5911, 0x5931), (0x6189, 0x618b), (0x6191, 0x6193),
        (0x6219, 0x621b), (0x6223, 0x6227), (0x6225, 0x622d), (0x6229, 0x622b),
        (0x6231, 0x6233), (0x6245, 0x62c5), (0x6249, 0x624b), (0x6251, 0x6253),
        (0x6289, 0x628b), (0x6291, 0x6293), (0x6311, 0x6313), (0x6445, 0x6447),
        (0x6449, 0x644d), (0x6451, 0x6453), (0x6489, 0x648b), (0x6491, 0x6493),
        (0x6511, 0x6513), (0x6889, 0x688b), (0x6891, 0x6893), (0x6911, 0x6913),
        (0x7111, 0x7113), (0x8889, 0x8a8b), (0x8893, 0x8897), (0x8895, 0x889d),
        (0x8899, 0x889b), (0x88a3, 0xa8a3), (0x88a5, 0x88b5), (0x88a9, 0x88ab),
        (0x88c5, 0x88c7), (0x88c9, 0x88cb), (0x8913, 0x8933), (0x8915, 0x8917),
        (0x8923, 0x8927), (0x8925, 0x892d), (0x8929, 0x892b), (0x8945, 0x8947),
        (0x8989, 0x898b), (0x8a23, 0x8a27), (0x8a25, 0x18a25), (0x8a29, 0x8a2d),
        (0x8a45, 0x8a47), (0x8a49, 0x8a4d), (0x8c49, 0x8c4b), (0x8c89, 0x8c8b),
        (0x9123, 0x9127), (0x9125, 0x9325), (0x9129, 0x912b), (0x9145, 0x9345),
        (0x9149, 0x914b), (0x9225, 0x19225), (0x9229, 0x922d), (0x9245, 0x9a45),
        (0x9249, 0x49249),
    ]),
]

def require(condition, message):
    if not condition:
        raise ValueError(message)

def verify(n, d, target, seeds):
    full = (1 << n) - 1
    def rotate(mask, shift):
        return ((mask << shift) | (mask >> (n-shift))) & full
    intervals = set()
    distinct_orbits = set()
    for a, b in seeds:
        require(0 <= a <= full and 0 <= b <= full, "mask range")
        require(a & b == a and a.bit_count() >= d, "lower endpoint")
        require(b.bit_count() == target, "upper endpoint")
        orbit = frozenset((rotate(a,t), rotate(b,t)) for t in range(n))
        require(orbit not in distinct_orbits, "duplicate orbit")
        distinct_orbits.add(orbit)
        intervals.update(orbit)
    multiplicity = bytearray(1 << n)
    for a, b in intervals:
        free, part = b ^ a, b ^ a
        while True:
            mask = a | part
            require(multiplicity[mask] == 0, "overlapping intervals")
            multiplicity[mask] = 1
            if part == 0:
```

```
                break
            part = (part-1) & free
    singletons = 0
    for mask in range(1 << n):
        rank = mask.bit_count()
        if rank < d:
            require(multiplicity[mask] == 0, "covered outside ideal")
        elif rank < target:
            require(multiplicity[mask] == 1, "uncovered small set")
        elif multiplicity[mask] == 0:
            multiplicity[mask] = 1
            singletons += 1
    expected = sum(comb(n,j) for j in range(d,n+1))
    require(sum(multiplicity) == expected, "wrong domain size")
    require(target == d + (n-d)//(d+1), "counting upper bound")
    return dict(n=n, d=d, sdepth=target, result="PASS",
                orbit_seeds=len(seeds),
                nontrivial_intervals=len(intervals),
                singleton_intervals=singletons,
                ideal_subsets=expected,
                lower_ranks=dict(sorted(Counter(
                    a.bit_count() for a,b in intervals).items())))

if __name__ == "__main__":
    print(json.dumps([verify(*case) for case in CASES], indent=2))
```